

# Building It: My Automation Pipeline, Step by Step

How the pieces connect, and where I'm hosting everything for free

This is the build log for the automation layer of my Retail Decision Automation Platform — the part that goes beyond a dashboard into an actual pipeline: data flowing in on a schedule, landing in a real warehouse, and triggering an alert on its own when something needs attention. I'm writing it the way I actually worked through it, including the one tool swap I made along the way.

## A note on the dashboard tool

I started with Streamlit for the dashboard layer, but ran into enough friction that I switched to **Grafana Cloud** instead. It has a forever-free tier — 10,000 metric series, 3 users, no credit card, no time limit — and a built-in PostgreSQL data source I could point straight at my warehouse. The part that actually mattered for this project: Grafana has native alerting that can post directly to Slack, which meant I didn't have to hand-roll all of my alert delivery in Python. I kept my own alert-check script for the parts of the logic that needed custom business rules, but let Grafana handle the visualization and the simpler threshold alerts.

## 1. Understanding the pieces before connecting them

Before wiring anything together, I laid out what each tool was actually responsible for, so I wouldn't end up with logic duplicated across three different places:

| Tool                | Job                                                                   |
|---------------------|-----------------------------------------------------------------------|
| GitHub Actions      | Runs my pipeline on a schedule — the automation trigger               |
| Python (pandas)     | Cleans and transforms the raw data                                    |
| Neon Postgres       | The warehouse — single source of truth everything else reads from     |
| Grafana Cloud       | Reads the warehouse live, shows dashboards, handles simple alerting   |
| Python alert script | Handles the custom business-rule checks Grafana's simple alerts can't |
| Slack webhook       | Where the alert message actually lands                                |

## 2. The build, step by step

### Step 1 — Get the data source ready

I used a public retail dataset to start (Kaggle's "Online Retail" set works well), with a small Python generator script I can also use to simulate "new" transactions arriving, so the pipeline has something fresh to process on each run instead of static data.

### Step 2 — Write the ETL script

One script, `etl.py`: reads the raw data, cleans it up (duplicates, nulls, type mismatches), calculates the fields I actually need for analysis (profit, margin), and writes the result into Postgres using `psycopg2`. I kept this script isolated and well-commented, since it's the piece I'll walk through in interviews.

### Step 3 — Stand up the warehouse

I created a free Postgres instance on Neon (Supabase is an equally good option). That gave me a connection string, which I stored as a GitHub Actions secret rather than anywhere in the code itself.

### Step 4 — Build the star schema

A one-time setup script created my `FactSales` table plus the dimension tables (`DimCustomer`, `DimProduct`, `DimDate`, `DimLocation`). This only needed to run once.

### Step 5 — Automate the pipeline

I added a GitHub Actions workflow file with a cron schedule. Every run checks out the repo, installs dependencies, pulls the database secret, and runs `etl.py`. This is the actual automated data pipeline — and because it's a GitHub Actions run, it's publicly visible proof that it works, not just a claim in my README.

### Step 6 — Connect the dashboard

Instead of Streamlit, I connected Grafana Cloud's built-in PostgreSQL data source directly to my Neon database and built the dashboard panels there. Because it queries the live database, it reflects new data automatically after every pipeline run — no

manual republishing.

### Step 7 — Add the automated decision logic

A second script, `check_alerts.py`, runs a SQL query against the warehouse checking a condition I define — for example, a product's margin dropping more than 15% week over week. When triggered, it can optionally call an LLM API to turn the flagged numbers into a short written recommendation, rather than just a raw number.

### Step 8 — Wire up alert delivery

For straightforward threshold breaches, I let Grafana's native alerting handle it and post to Slack directly. For the more custom checks — the ones with business logic Grafana can't express — `check_alerts.py` posts to the same Slack channel through an incoming webhook.

### Step 9 — Chain it into one workflow

My GitHub Actions YAML runs `etl.py` and then `check_alerts.py` in sequence on the same schedule. One trigger, one visible log, a full path from raw data to a Slack message.

## 3. How everything is connected



## 4. Where everything is hosted, and what it costs

| Piece                     | Where it lives                        | Cost                                 |
|---------------------------|---------------------------------------|--------------------------------------|
| Code, docs, workflow file | GitHub (public repo)                  | Free                                 |
| Scheduler / orchestration | GitHub Actions                        | Free (2,000 min/month)               |
| Database / warehouse      | Neon (or Supabase)                    | Free tier, persistent                |
| Dashboard + alerting      | Grafana Cloud                         | Free tier, forever, no card required |
| Alert delivery            | Slack (free workspace + webhook)      | Free                                 |
| Demo walkthrough          | Loom free tier or a GIF in the README | Free                                 |

Nothing in this stack costs money, and every piece is a real, running system I can point someone to — not a screenshot or a claim.

## 5. What I'd tell someone about it

Rather than saying "I built a dashboard," what I actually did was design a pipeline that pulls data automatically, keeps a live warehouse in sync, and flags problems on its own — with the visualization layer being just one consumer of that pipeline, not the whole project.